CARAT: Class-Aware Robust Aggregation via Hidden Task Certificates for Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning servers must aggregate client updates without observing client data, making them vulnerable to both untargeted model poisoning and targeted backdoor attacks. Existing robust aggregators mainly rely on update geometry, coordinate-wise robust statistics, or alignment with a single trusted reference direction. These signals can be brittle when benign clients are heterogeneous and adaptive attackers remain close to the benign update manifold. We present **CARAT**, a class-aware robust aggregation rule based on *hidden task certificates*. CARAT uses server-visible labeled reference data to instantiate multiple hidden, class-balanced micro-tasks and scores each client update by the loss decrease it induces on those tasks. It then combines robust norm clipping, common-radius evaluation, worst-task optimization through a soft minimum over certificate scores, coordinate-rank anomaly penalties, and capped-simplex weight optimization with temporal regularization. This design favors updates that are broadly useful across semantic sub-tasks while suppressing clients that remain structurally extreme in rank space. In a verified FedAvg/CIFAR100/FangAttack run, CARAT substantially improves over Mean and assigns zero weight to the attacker coalition in late rounds. Because the strongest completed CARAT run currently available disables test-time evaluation, this evidence should be read as an initial empirical validation rather than a complete benchmark; nevertheless, it supports the promise of task-aware aggregation and motivates completion of the clean-accuracy and ASR benchmark now underway.

1 Introduction

Federated learning (FL) enables many clients to train a shared model without centralizing their raw data [1]. This decentralization is attractive for privacy, governance, and communication, but it also leaves the server in a fundamentally asymmetric position: it must update the global model using client messages that it cannot directly verify. Poisoning attacks exploit exactly this asymmetry. Malicious clients can slow or prevent convergence, bias the learned decision boundary, or implant triggers that activate attacker-chosen behavior at test time [2–6]. In practice, the security of FL is therefore tightly coupled to the aggregation rule.

The central difficulty is that malicious behavior and benign heterogeneity can look superficially similar. Under non-i.i.d. partitions, benign clients legitimately differ in direction, norm, sparsity, and class emphasis. As a result, an update can appear unusual either because it is harmful or because it reflects a different local data distribution. Many existing defenses rely on signals that are too coarse to resolve this ambiguity reliably. Pairwise distances and geometric centrality can misclassify benign dispersion as adversarial deviation; coordinate-wise robust statistics suppress

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

only marginal extremes and ignore cross-coordinate structure; trusted-reference schemes reduce the server's auxiliary information to a single global direction, which can miss failures concentrated on particular classes or semantic regions.

This observation motivates the main question behind CARAT: *can the server evaluate an update by testing whether it is consistently useful across hidden semantic sub-tasks, rather than only by testing whether it looks similar to other updates?* We answer this question with **CARAT**, *Class-Aware Robust Aggregation via Hidden Task Certificates*. CARAT assumes server-side access to labeled reference data and uses that data to sample hidden, class-balanced micro-tasks. Each client update is then scored by the loss reduction it induces across these tasks. The resulting certificate matrix gives the server a finer-grained notion of utility: an update should be trusted not merely because it looks central, but because it remains broadly helpful across multiple hidden slices of the learning problem.

CARAT complements this task-aware signal with two additional ingredients. First, it uses robust clipping and common-radius evaluation so that certificate comparisons are not dominated by raw magnitude. Second, it adds a coordinate-rank anomaly penalty to capture whether a client remains structurally extreme even after clipping. Aggregation weights are then learned on a capped simplex through a worst-task objective with temporal regularization. The overall goal is to preserve benign diversity while suppressing updates that are either semantically brittle or structurally anomalous.

This design places CARAT between trust-based aggregation and task-aware validation. Compared with FLTrust, it expands trusted data into many hidden probes rather than a single trusted direction. Compared with heterogeneity-aware schemes such as bucketing, it does not try to homogenize clients before aggregation; instead, it directly asks whether each update is useful across hidden task slices. Compared with inspection-oriented defenses such as FLAME and DeepSight, it intervenes at aggregation time rather than primarily through post-hoc filtering. In this sense, CARAT does not merely introduce another distance rule. It changes the server's validation signal from update similarity to hidden-task utility.

Contributions. Our contributions are as follows.

- We introduce hidden-task certificates as a task-aware server-side validation signal for robust FL aggregation, using multiple class-balanced micro-tasks instead of a single trusted reference direction.
- We formulate CARAT as a capped-simplex weight optimization problem that jointly enforces worst-task utility, rank-space anomaly suppression, and temporal regularization.
- We make the implementation assumptions explicit, most importantly the server-side access to labeled data from which the hidden-task probe pool is constructed.
- We report the current repository-backed empirical evidence and separate verified results from the larger clean-accuracy and ASR benchmark suite that is still being completed.

2 Method

2.1 Problem setting

We consider synchronous FL with n clients and a global parameter vector $w_r \in \mathbb{R}^d$ at round r. At each round, client i returns a message u_i^r, which the server maps to a vector-form update $\Delta_i^r \in \mathbb{R}^d$. For FedAvg, Δ_i^r can be viewed as the client model delta relative to the current global model; for gradient-style optimizers, it is the client update itself. The server must aggregate $\{\Delta_i^r\}_{i=1}^n$ into a single global step while a fraction ρ of clients may be malicious and may coordinate their behavior.

Server knowledge. CARAT assumes the server has access to labeled reference data. From this reference set it builds a probe pool and repeatedly samples hidden, class-balanced validation tasks. This is a materially stronger assumption than fully data-free robust aggregation and should be treated as part of the method definition rather than as an incidental implementation detail.

Goal. The goal is to assign aggregation weights that preserve benign diversity while suppressing updates that are either semantically harmful or structurally anomalous. The key challenge is to

distinguish malicious deviation from legitimate non-i.i.d. variation without relying only on geometric similarity.

2.2 CARAT overview

CARAT is organized around three ideas. First, it validates updates through hidden task utility rather than through client-to-client similarity alone. Second, it emphasizes *worst-task* performance, reflecting the intuition that an update should not be trusted if it helps on average but damages a subset of hidden semantic slices. Third, it augments semantic utility with a structural anomaly signal in rank space.

Operationally, each round has three stages. The server first places updates into a fair evaluation geometry by clipping and rescaling them. It then scores each client using hidden-task certificates and coordinate-rank anomaly penalties. Finally, it solves a capped-simplex optimization problem whose solution defines the aggregation weights. A crucial design choice is that normalized updates are used only for *evaluation*, whereas clipped updates are used for the actual model step. CARAT therefore separates evaluation geometry from update geometry.

2.3 Hidden-task and anomaly scoring

Let $\mathcal{D}^{\mathrm{ref}}$ denote the server reference data and let $\mathcal{P}$ denote a probe pool sampled from it. At round r, CARAT samples T hidden tasks from $\mathcal{P}$, with each task constructed to be approximately class-balanced. This design is the source of CARAT's class-aware behavior: a malicious update may look globally plausible while degrading only a subset of classes, whereas balanced hidden tasks make such class-selective failures easier to detect.

Clipping and common-radius evaluation. Before hidden-task evaluation, CARAT robustly clips client updates. Let $\|\Delta_i\|_2$ denote the norm of client i's update. In our implementation, the clipping threshold τ_r is estimated by a MAD-based rule:

$$\tau_r = \mathrm{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \mathrm{MAD}(\{\|\Delta_i\|_2\}), \tag{1}$$

and the clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{2}$$

CARAT then rescales each clipped update to a common evaluation radius q_r:

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{3}$$

where q_r is chosen as a quantile of the positive clipped norms. This normalization removes pure scale advantages during hidden-task comparison. Importantly, it does *not* define the final server step; it only creates a common evaluation geometry in which client updates can be compared more fairly. Appendix Proposition A.3 formalizes the invariance induced by this step.

Hidden-task certificates. For each hidden task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{4}$$

Here, "certificate" denotes an internal task-level utility score rather than a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Stacking these scores yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because different tasks may have different intrinsic difficulties and baseline loss scales, CARAT robustly standardizes each column of C before optimization. This preserves within-task ordering while making scores comparable across tasks.

Rank-space anomaly penalties. Hidden-task utility alone is not sufficient: an attacker may overfit to the probe tasks or align with a subset of them while still remaining structurally atypical. CARAT therefore augments semantic utility with a coordinate-rank anomaly signal.

For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, CARAT ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$r_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{5}$$

CARAT repeats this computation over several random coordinate subsets and aggregates the resulting values robustly. A client receives a large anomaly score if its update remains persistently extreme across many coordinates. After robust standardization, only positive deviations are retained as penalties. The signal is intentionally asymmetric: suspicious extremeness is penalized, whereas unusual centrality is not explicitly rewarded.

2.4 Weight optimization and aggregation

CARAT chooses aggregation weights $\alpha \in \mathbb{R}^n$ on the capped simplex

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{6}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. This cap prevents any single client from dominating the aggregate and guarantees a minimum support size. Appendix Proposition A.1 states the corresponding feasibility and support guarantees.

Let $r \in \mathbb{R}^n$ denote the positive rank penalties and let α^{prev} be the previous round's weights. CARAT solves

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{7}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{8}$$

The softmin term gives a smooth approximation to worst-task utility: large β places more emphasis on the weakest hidden tasks, while finite β preserves optimization stability. The second term penalizes structurally extreme clients, and the third term regularizes the solution toward the previous round to reduce unnecessary oscillation. Appendix Proposition A.2 records the basic bounds and gradient form of the softmin term.

Eq. (7) yields a clear selection principle. A client should retain influence only if it is useful across hidden tasks *and* not persistently extreme in rank space. The temporal prior does not define trust by itself; instead, it resolves ambiguity among utility-equivalent feasible solutions and discourages gratuitous weight oscillation. Appendix Proposition A.4 formalizes this role.

Final aggregation and complexity. After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\mathrm{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{9}$$

This yields the final server step. The final update therefore inherits the robustness benefits of clipping while using weights learned from the richer evaluation geometry defined above.

Relative to standard aggregation, the main additional cost comes from evaluating n candidate updates over T hidden tasks and computing rank penalties on coordinate subsamples. Appendix Proposition A.5 summarizes the resulting per-round complexity. In practice, the probe-model forward passes dominate, while the rank and optimization stages are lower-order terms with tunable budgets.

3 Experiments

3.1 Experimental setup

Datasets and models. All paper-side experiment specs use FedAvg on CIFAR100 with a ResNet-18 backbone. We intentionally fix the dataset-model pair across the clean-reference, untargeted-poisoning, backdoor, and ablation suites so that changes in robustness can be attributed to the defense

Defense	Train acc. @299	Train loss @299	Mean round time (s)	Total runtime
Mean	0.5699	1.5187	1.66	8m25s
TriGuardFL	**0.6385**	1.2932	7.37	37m07s
CARAT	0.6286	**1.2748**	9.49	47m37s

Table 1: Completed pilot evidence from repository logs for FedAvg on CIFAR100 under i.i.d. FangAttack. These are training-only diagnostics because that pilot configuration disables test-time evaluation.

rather than to architecture changes. CIFAR100 is particularly relevant for CARAT because the method evaluates client updates across class-balanced hidden tasks; a larger class space makes class-selective failure modes more visible than in low-class-count benchmarks. The strongest completed legacy CARAT pilot also uses the same CIFAR100/ResNet-18 pair, so its evidence is directly comparable to the ongoing benchmark suite.

Attacks and compared defenses. We use separate suites for clean, untargeted, and backdoor evaluation. The clean-reference suite uses `NoAttack` and compares {Mean, FLTrust, MultiKrum, CARAT}. The main untargeted suite evaluates {ALIE, MinMax, FangAttack} against {Mean, FLTrust, MultiKrum, CARAT}. The backdoor suite evaluates {BadNets, Neurotoxin} against {Mean, FLTrust, FLAME, DeepSight, CARAT}. In addition, the strongest finished legacy pilot available today compares Mean, TriGuardFL, and CARAT under i.i.d. FangAttack. We continue to report that pilot because it is currently the only completed CARAT run with end-to-end logs strong enough to anchor a concrete main-text table.

FL setting. CARAT is implemented as a server aggregator in FLPoison and operates on the vector-form client deltas produced by the runtime. All committed paper-side specs use synchronous FedAvg with 20 clients, 300 communication rounds, local epoch count 4, batch size 64, and learning rate 0.05. The clean-reference suite uses attacker fraction 0, the main untargeted suite sweeps attacker fractions $\{0.2, 0.4\}$, and the backdoor suite uses attacker fraction 0.2. Heterogeneity is represented by i.i.d. partitions and Dirichlet non-i.i.d. splits: `clean_reference.yaml` uses $\alpha \in \{0.1, 0.5\}$, `main_untargeted.yaml` uses $\alpha \in \{0.1, 0.5\}$, and `backdoor.yaml` uses $\alpha = 0.5$. Each paper-side suite runs three repeats, yielding effective seeds 42, 43, and 44. The default CARAT hyperparameters are: probe pool size 512, 8 hidden tasks, 4 probe samples per class, probe batch size 128, probe-radius quantile 0.5, MAD clipping with $k = 2.5$, default attacker prior $\hat{\rho} = 0.2$, certificate temperature $\beta = 12.0$, rank weight 0.20, prior weight 0.05, 4096 sampled coordinates, 5 rank subsamples, 60 optimization steps, and optimizer learning rate 0.5.

Metrics. For clean and untargeted runs, `metrics_exp*.csv` records per-round training and test accuracy and loss. For backdoor runs, we report clean test accuracy together with attack success rate (ASR); ASR and related attack-specific diagnostics are parsed from the detailed run logs. To capture computational cost, we also report mean round time and end-to-end runtime when those quantities are available. One important caveat is that the strongest completed legacy CARAT pilot was produced from an earlier base config with `evaluate=false`. Its main-text evidence is therefore limited to training accuracy, training loss, and runtime, whereas all paper-side benchmark suites are configured to record test-time metrics.

3.2 Current empirical results

Table 1 summarizes the strongest *completed CARAT evidence* currently verified end-to-end in the repository: a single-seed pilot FedAvg run on CIFAR100 with an i.i.d. split, 20 clients, 20% attackers (4 malicious client slots), ResNet-18, 300 rounds, and FangAttack. The available logs cover Mean, TriGuardFL, and CARAT under the same base training configuration. Because that pilot configuration sets `evaluate=false`, the table reports *training-only* diagnostics and runtime rather than clean-test accuracy or ASR.

CARAT substantially improves over simple averaging in the verified run. Relative to Mean, it increases round-299 training accuracy by about 5.9 points and reduces training loss by about 0.24. At the same time, the current evidence does *not* support a broader claim of uniform dominance across

strong baselines: on this single training-only run, TriGuardFL finishes with slightly higher training accuracy, whereas CARAT attains the lowest training loss. We therefore interpret Table 1 as evidence that the method is functioning as intended, not as a complete comparative benchmark.

An evaluation-enabled main benchmark is now running in the repository. In the current repository snapshot dated April 16, 2026, the first four completed tasks from `main_untargeted.yaml` all come from the i.i.d. ALIE, 20% attacker slice: three Mean repeats and one FLTrust repeat. The three Mean runs finish at epoch 299 with eval accuracy 57.55%, 57.97%, and 57.39% and eval loss 1.579, 1.544, and 1.554; the completed FLTrust run finishes at 47.25% eval accuracy and 1.937 eval loss. These logs confirm that the new benchmark path is recording test-time metrics as intended, but the corresponding CARAT tasks in this suite have not finished yet, so they do not yet support a CARAT comparison.

The most diagnostic CARAT-specific signal is the behavior of the learned weights. In this benchmark, the coordinator assigns the first `num_adv` worker slots to attackers, so clients 0–3 are malicious. Late in training, CARAT repeatedly logs weight vectors of the form

$$\alpha = [0, 0, 0, 0, 0.0625, 0.0625, \ldots, 0.0625],$$

which means the optimizer places zero mass on the entire malicious coalition while distributing the remaining mass uniformly over benign clients. At epoch 298, for example, the log records task scores $[0.2879, 0.2784, 0.2859, 0.2781, 0.2533, 0.3009, 0.2930, 0.2876]$, a softmin value of 0.2820, evaluation radius 18.1484, and clip norm 18.3227. These quantities indicate that the implementation is doing more than hard filtering: it is identifying weak hidden tasks under a common scoring radius and then applying the optimized weights to the clipped updates used for aggregation.

These observations are encouraging, but they still do not establish improved clean-test accuracy, reduced ASR, or robustness across non-i.i.d. regimes for CARAT. The remaining empirical gap is no longer instrumentation but coverage: the evaluation-enabled CARAT benchmark has started, yet its CARAT rows, non-i.i.d. tasks, backdoor suite, and ablations are not complete.

3.3 Ablation design

The cleanest ablations are those already supported by the current configuration interface. The first ablation package should isolate:

- probe pool size,
- number of hidden tasks,
- per-class probe samples,
- clipping rule, clipping threshold, and evaluation-radius quantile,
- rank penalty weight λ_r,
- prior weight λ_p,
- softmin temperature β,
- rank subsampling budget and optimizer steps.

These ablations are directly supported by the preset groups summarized in Table 2. This design already supports a concrete methodological question: how much of CARAT's behavior comes from task construction, clipping and normalization, rank penalties, or temporal regularization?

Two additional structural ablations would strengthen the final paper but require small code patches rather than simple config changes: replacing the softmin objective with average utility, and bypassing hidden-task certificates entirely. The current implementation does not expose those switches as top-level parameters, so the paper should not imply that those ablations have already been run.

The final empirical narrative should first establish robustness with clean-test metrics, then evaluate backdoor settings using both ASR and clean utility, and finally use the supported ablations above to determine whether CARAT's gains come primarily from hidden-task validation, rank-space filtering, or their interaction.

Factor	Directly supported config keys
Probe construction	`num_probe_pool`, `num_probe_tasks`, `probe_samples_per_class`
Clipping / normalization	`clip_method`, `clip_k`, `clip_percentile`, `probe_radius_quantile`
Objective shaping	`certificate_beta`, `rank_weight`, `prior_weight`, `rho_hat`
Rank estimation	`rank_num_coords`, `rank_num_subsamples`
Optimizer budget	`optimizer_steps`, `optimizer_lr`

Table 2: Ablation axes already exposed by the current CARAT implementation. This is the safest starting point for a code-aligned ablation section.

3.4 Current scope and limitations

A core methodological limitation remains the server-side data assumption already stated in the Method section: in the current code path, the probe pool is sampled from a server-visible labeled training dataset rather than from a tiny external reference set. This assumption matters when CARAT is compared to fully data-free robust aggregators.

Computational cost is a second practical limitation. Compared with lightweight rules such as Mean, CARAT adds repeated hidden-task forward passes, rank subsampling, and projected optimization. On the current FangAttack/CIFAR100 pilot, the mean round time is 9.49s for CARAT versus 1.66s for Mean and 7.37s for TriGuardFL. Any final empirical case must therefore justify CARAT's extra server-side cost with corresponding clean-test or ASR gains, not only with training-time diagnostics.

Finally, empirical coverage is still incomplete. The strongest completed CARAT evidence is a single-seed, training-only FangAttack pilot. An evaluation-enabled benchmark suite that includes CARAT is now committed and running, but the CARAT rows, non-i.i.d. cases, backdoor tasks, and adaptive-attack studies are not yet complete. This means the current manuscript supports a methodology claim and an initial empirical result, but not yet a full benchmark claim.

4 Related Work

The most relevant prior work differs primarily in *what signal* it uses to decide whether a client should influence the global model. This perspective is useful for evaluating CARAT because the method does not mainly contribute a new distance metric or a new filter; it changes the unit of validation from client-to-client similarity to client-to-task utility.

Similarity-based robust aggregation. Classical Byzantine-robust rules infer benignness from centrality. Krum and related geometry-based methods favor updates that remain close to other clients under pairwise distances [7, 8], while coordinate-wise median and trimmed mean suppress marginal extremes independently [9]. These methods are appealing because they require little server-side prior knowledge. Their weakness is that they implicitly assume benign updates should look mutually similar. Under non-i.i.d. data, that assumption becomes fragile: benign clients may be dispersed for legitimate reasons, and adaptive attackers may remain close to the benign manifold while still harming a subset of classes or semantic regions.

Heterogeneity-aware and reference-data defenses. Two important responses to this limitation are to reshape the client population or to inject trusted server knowledge. Bucketing reduces heterogeneity by averaging clients before robust aggregation [10]. FLTrust instead uses a trusted server dataset to compute a reference direction and reweights clients by alignment with that anchor [11]. Other defenses, including FoolsGold and FLDetector, add historical or representation-level diagnostics rather than relying purely on instantaneous geometry [12, 15]. CARAT is closest to this family because it also exploits server-side labeled data. The key difference is that it does not compress trusted information into one global direction. Instead, it expands that information into many hidden validation tasks, allowing the server to ask whether an update is broadly useful rather than merely well aligned.

Adaptive poisoning and backdoor defenses. Modern attacks are explicitly optimized against robust aggregators. FangAttack, MinMax, MinSum, and ALIE are representative untargeted attacks

designed to evade simple geometric or statistical filters [2, 3, 16]. On the backdoor side, model replacement, edge-case attacks, and durable attacks such as Neurotoxin show that an update can remain outwardly plausible while still encoding highly targeted malicious behavior [4–6]. Backdoor-oriented defenses such as FLAME, DeepSight, and CRFL respond with clustering, inspection, clipping, or smoothing mechanisms [13, 14, 17]. Taken together, this literature suggests that one-dimensional defenses are increasingly brittle: when the server validates only one property, adaptive attackers can often optimize against exactly that property.

Positioning of CARAT. CARAT is best understood as a task-aware aggregation rule for the regime where benign heterogeneity and adaptive poisoning interact. Compared with FLTrust, it uses trusted data to instantiate many hidden semantic probes rather than one trust anchor. Compared with bucketing, it does not pre-average clients to reduce dispersion; it directly evaluates each client on hidden task slices. Compared with FLAME and DeepSight, it intervenes at aggregation time instead of relying primarily on post-hoc inspection or clustering. The intended contribution is therefore not another robustness heuristic layered on top of update similarity, but a different validation principle: a useful client update should remain helpful across hidden sub-tasks, not merely look centrally located.

5 Conclusion

CARAT combines hidden class-balanced task certificates, rank-space anomaly suppression, and capped-simplex weight optimization into a single aggregation rule. The central claim of the paper is not merely that robust aggregation should filter outliers, but that it should validate client updates against multiple hidden semantic slices of the task. This perspective is what allows CARAT to separate benign diversity from malicious inconsistency more directly than geometry-only or single-reference defenses.

The current repository already supports a concrete methodology claim and an initial empirical claim: in a completed CIFAR100/FangAttack pilot, CARAT suppresses attacker weights and substantially improves over Mean, albeit with higher runtime and without yet surpassing every strong baseline on that single seed. The immediate next step is to complete the evaluation-enabled untargeted, backdoor, and ablation suites so that the final paper reports clean accuracy, ASR, runtime overhead, and multi-seed variance under both i.i.d. and non-i.i.d. settings.

If those experiments confirm the pilot pattern, CARAT would offer a practically relevant path toward more reliable FL aggregation in settings where class-selective failures matter, including safety-sensitive deployments. Even if the gains turn out to be conditional, the hidden-task view still provides a useful organizing principle for designing and stress-testing future FL defenses.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[3] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[4] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020.

[5] Hongyi Wang, Kartik Sreenivasan, Shashank Rajput, Harit Vishwakarma, Saurabh Agarwal, Jy-yong Sohn, Kangwook Lee, and Dimitris Papailiopoulos. Attack of the tails: Yes, you really can backdoor federated learning. In *Advances in Neural Information Processing Systems*, 2020.

[6] Zhengming Zhang, Ashwinee Panda, Linyue Song, Yaoqing Yang, Michael Mahoney, Prateek Mittal, Ramchandran Kannan, and Joseph Gonzalez. Neurotoxin: Durable backdoors in

federated learning. In *International Conference on Machine Learning*, pages 26429–26446, 2022.

[7] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[8] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[9] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[10] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[11] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[12] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[13] Thien Duc Nguyen, Phillip Rieger, Hieu Yalame, Hauke Möllering, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. FLAME: Taming backdoors in federated learning. In *USENIX Security Symposium*, 2022.

[14] Phillip Rieger, Thien Duc Nguyen, Hauke Möllering, Hieu Yalame, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection. In *Network and Distributed System Security Symposium*, 2022.

[15] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[16] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[17] Chulin Xie, Minghao Chen, Pin-Yu Chen, and Bo Li. CRFL: Certifiably robust federated learning against backdoor attacks. In *International Conference on Machine Learning*, pages 11372–11382, 2021.

A Theoretical Analysis

Proposition A.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{A}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{A}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof sketch. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition A.2 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta}\log\left(\frac{1}{T}\sum_{t=1}^{T} e^{-\beta s_t}\right).$$

Then

$$\min_t s_t \leq \operatorname*{softmin}_{\beta}(s) \leq \frac{1}{T}\sum_{t=1}^{T} s_t.$$

Moreover, if $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T}\exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof sketch. The lower and upper bounds follow from standard log-sum-exp inequalities: the exponential average is bounded below by the largest exponential term and above by the arithmetic mean. Monotonicity in each coordinate is immediate. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule then gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition A.3 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a\bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (3) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \ldots, T\}.$$

Proof sketch. Eq. (3) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a\bar{\Delta}_j}{\|a\bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (4) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition A.4 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{A}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $r^\top \alpha = r^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p\left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2\right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)})\right),$$

every iterate remains in $\mathcal{A}(\hat{\rho})$.

Proof sketch. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1-\hat{\rho})n)$. □

Proposition A.5 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof sketch. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the current implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on assumptions. The theoretical object D^{ref} in the main text should be read as the probe pool *after* subsampling. The code, however, obtains that pool by sampling from the full server-visible training dataset. Any final formal discussion should keep this distinction explicit, because it materially strengthens the server-side assumption relative to a fully data-free defense.

B Algorithm

Inputs. Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server-visible training dataset, and CARAT hyperparameters.

Persistent state. CARAT maintains two pieces of state across rounds: a balanced probe pool sampled at initialization from the server-visible training dataset, and the previous-round weight vector α^{prev}, which serves as the temporal prior in the current round.

Initialization. At the first round, the implementation initializes α^{prev} to the uniform vector. This makes the initial round unbiased across clients while still allowing later rounds to exploit temporal consistency once CARAT has observed several optimization trajectories.

Procedure.

1. Convert client messages to vector-form deltas Δ_i^r using the common update-preparation utility.
2. Compute client norms, choose a clipping threshold with the configured MAD or percentile rule, and form clipped deltas $\bar{\Delta}_i^r$.
3. Choose an evaluation radius from the configured quantile of positive clipped norms and normalize clipped deltas to obtain $\tilde{\Delta}_i^r$.
4. Sample class-balanced hidden tasks from the fixed probe pool for round r; if strict balanced sampling fails, fall back to replacement and, if necessary, to a random subset rather than aborting the round.
5. Load the current global weights into the server-side probe model, evaluate baseline hidden-task losses, and compute certificate scores $c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i^r)$ for all clients and tasks.
6. Robustly standardize certificate columns, compute rank-space penalties from coordinate subsamples of $\bar{\Delta}_i^r$, and retain only positive standardized rank deviations.
7. Resolve $\hat{\rho}$, initialize weights from α^{prev}, and optimize

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2$$

   by projected gradient ascent on the capped simplex.
8. Update α^{prev} with the optimized solution.
9. Aggregate the clipped deltas $\bar{\Delta}_i^r$ with the optimized weights and return the resulting global step.

Round invariants and failure handling. Two implementation choices are important for robustness. First, the optimizer never leaves the capped simplex, so every intermediate and final weight vector remains feasible. Second, probe-task construction is fail-soft rather than fail-stop: if strict class-balanced sampling is impossible, the code falls back to replacement or to a random subset, ensuring that the round can still proceed with a well-defined certificate matrix.

Outputs. Aggregated server update, updated temporal prior α^{prev}, and diagnostic logs including learned weights, task scores, softmin focus, rank penalties, evaluation radius, clipping threshold, and the identities of the hidden probe tasks used in the current round.

C Additional Experiments

This section summarizes the current verified experimental scope together with the next empirical extensions already supported by the repository structure.

Currently verified run. The strongest directly verified completed CARAT run in the repository uses FedAvg, CIFAR100, ResNet-18, an i.i.d. partition, 20 clients, 4 attackers, FangAttack, 300 rounds, learning rate 0.05, local epochs 4, and `evaluate=false`. The main text reports the final training metrics and runtime of this run together with the corresponding Mean and TriGuardFL baselines. In the current repository snapshot dated April 16, 2026, the evaluation-enabled `main_untargeted.yaml` sweep has also started producing ALIE baseline logs, but no CARAT task from that sweep is complete yet.

Supported benchmark matrix. The committed `main_untargeted.yaml` spec already expands to 216 tasks over i.i.d. and non-i.i.d. splits with Dirichlet $\alpha \in \{0.1, 0.5\}$, attacker fractions $\{0.2, 0.4\}$, untargeted attacks {ALIE, MinMax, FangAttack}, defenses {Mean, FLTrust, MultiKrum, CARAT}, and three repeats. Companion specs `clean_reference.yaml`, `backdoor.yaml`, and `ablations.yaml` cover clean baselines, backdoor evaluation, and component analysis.

Next additions.

1. Complete the running `main_untargeted.yaml` sweep, prioritizing the CARAT rows and reporting multi-seed means and variances for clean test accuracy and loss.
2. Extend the verified runs from the current i.i.d. slice to the non-i.i.d. Dirichlet settings already present in `main_untargeted.yaml`.
3. Complete the committed `backdoor.yaml` suite so that the paper reports both clean utility and ASR.
4. Run the config-supported ablations in Table 2, prioritizing the number of hidden tasks, the rank and prior weights, and the clipping controls.
5. Report multi-seed means and variances, plus defense runtime overhead, before making any strong comparative claim against other robust aggregators.